Bahria University, Islamabad

Department of Software Engineering

Object Oriented Programming Lab (Spring-2026)

Instructor: Engr. Muhammad Faisal Zia

Student:

Enrollment:

Lab Journal: 04

Date:

Task No:	Task Wise Marks		Documentation Marks		Total Marks (25)
	Assigned	Obtained	Assigned	Obtained	
1					
2					
3					
4					
5					

Comments:

Signature

Lab # 4

Friend Functions and Friend Classes in C++

Objective:

The objective of this lab is to understand the concept of **friend functions** and **friend classes** in C++. By the end of this lab, you should be able to:

1. Define and use friend functions to access private and protected members of a class.
2. Define and use friend classes to access private and protected members of another class.
3. Apply friend functions and friend classes in real-world scenarios like mathematical operations and factorial calculations.

Introduction

One of the important concepts of OOP is data hiding, i.e., a non-member function cannot access an object's private or protected data. But sometimes this restriction may force programmers to write long and complex code. So, there is a mechanism built in C++ programming to access private or protected data from non-member functions. This is done using a friend function or/and a friend class.

1. Friend Functions

- A friend function is a non-member function that has access to the private and protected members of a class.
- It is declared inside the class using the **friend** keyword.
- It is defined outside the class as a normal function.
- The compiler knows a given function is a friend function using the keyword friend. For accessing the data, the declaration of a friend function should be made inside the body of the class (can be anywhere inside class either in private or public section) starting with keyword friend. Declaration of friend function in C++

Syntax

```cpp
class ClassName {
    friend returnType functionName(parameters);
};
```

Example: Working of a Friend Function

```cpp
/* C++ program to demonstrate the working of friend function.*/
#include <iostream>
using namespace std;

class Distance {
private:
    int meter;

public:
    Distance() : meter(0) {} // Constructor to initialize meter to 0
```

```
        // Declare friend function
        friend int addFive(Distance);
    };

    // Friend function definition
    int addFive(Distance d) {
        d.meter += 5; // Access private data member
        return d.meter;
    }

    int main() {
        Distance D;
        cout << "Distance: " << addFive(D) << endl;

        return 0;
    }
```

```
Distance: 5
```

Example: Add member of two different Classes

```
// Add members of two different classes using friend functions
#include <iostream>
using namespace std;

// forward declaration
class ClassB;

class ClassA {

public:
    // constructor to initialize numA to 12
    ClassA() : numA(12) {}
private:
    int numA;
    // friend function declaration
    friend int add(ClassA, ClassB);
};

class ClassB {

public:
    // constructor to initialize numB to 1
    ClassB() : numB(1) {}

private:
    int numB;

    // friend function declaration
    friend int add(ClassA, ClassB);
};

// access members of both classes
int add(ClassA objectA, ClassB objectB) {
    return (objectA.numA + objectB.numB);
}

int main() {
    ClassA objectA;
```

```
    ClassB objectB;
    cout << "Sum: " << add(objectA, objectB);
    return 0;
}
```

```
Sum: 13
```

One thing to notice here is the friend function inside ClassA is using the ClassB. However, we haven't defined ClassB at this point.

```
 // inside classA
```

```
// inside classA
friend int add(ClassA, ClassB);
```

For this to work, we need a forward declaration of ClassB in our program.

```
// forward declaration
class ClassB;
```

2. Friend Class

- A **friend class** is a class that has access to the private and protected members of another class.
- It is declared inside the class using the `friend` keyword.
- When a class is declared a friend class, all the member functions of the friend class become friend functions.
- Since ClassB is a friend class, we can access all members of ClassA from inside ClassB.
- *<u>However, we cannot access members of ClassB from inside ClassA. It is because friend relation in C++ is only granted, not taken.</u>*

Syntax

```cpp
class ClassB;

class ClassA {
   // ClassB is a friend class of ClassA
   friend class ClassB;
   ... .. ...
}

class ClassB {
   ... .. ...
}
```

Example: C++ Friend Class

```
// C++ program to demonstrate the working of friend class

#include <iostream>
using namespace std;
```

```cpp
// forward declaration
class ClassB;

class ClassA {
private:
    int numA;

    // friend class declaration
    friend class ClassB;

public:
    // constructor to initialize numA to 12
    ClassA() : numA(12) {}
};

class ClassB {
private:
    int numB;

public:
    // constructor to initialize numB to 1
    ClassB() : numB(1) {}

    // member function to add numA
    // from ClassA and numB from ClassB
    int add() {
        ClassA objectA;
        return objectA.numA + numB;
    }
};
int main() {
    ClassB objectB;
    cout << "Sum: " << objectB.add();
    return 0;
}
```

```
Sum: 13
```

Conclusion:

In this lab, we learned:

1. How to define and use friend functions to access private and protected members of a class.
2. How to define and use friend classes to access private and protected members of another class.
3. Practical applications of friend functions and friend classes in mathematical operations and factorial calculations.

Lab Tasks

Task 01: Create two classes: alpha and beta (friend class). The constructors in these classes initialize their single data item to fixed values (3 in alpha and 7 in beta).

Create a friend function that has access to both private data members of these classes.
An object of each class is passed as an argument to the friend function. This friend function adds the data items and returns the sum. The output of the program should be like:
Output should be like this

```
Microsoft Visual Studio Debug Console
Sum of data members: 10

C:\Users\USER\source\repos\Labo5 T11\x64\Debug\Labo5 T11.exe (pro
cess 7912) exited with code 0.
To automatically close the console when debugging stops, enable T
ools->Options->Debugging->Automatically close the console when de
bugging stops.
Press any key to close this window . . .
```

Task 02: Write a program to display the sum, difference, product and division of two numbers using friend function. Create a class alpha. Declare four friend functions:

friend double sum (alpha)

friend double sub(alpha)

friend double mul(alpha)

friend double div(alpha)

Display the result. The output of the program should be like:

```
C:\Users\USER\source\repos\La...
Enter two numbers: 35 25
Sum: 60
Difference: 10
Product: 875
Division: 1.4

Press any key to continue...
```

Task 03: Create a class Student that stores student information. Each object of the class should contain private data members to store student details. Provide the following memberships:

1. A constructor to initialize the studentID, name, and totalMarks.
2. A member function displayBasicInfo() to show only the studentID and name.
3. A **friend function** compareTotalMarks() that can directly access private totalMarks of two students and display which student has higher total marks.
4. A **friend class** Teacher that can access all private student information and has a member function updateTotalMarks() to modify student total marks